

FRED TALKS

25th June 2026

CONTENTS

Getting Real About Distributed System Reliability

BOREDANDROID · 2792 WORDS

Test Coverage Won't Save You

JENN COOPER · 1991 WORDS

A non-anthropomorphized view of LLMs

HALVAR.FLAKE · 1478 WORDS

Getting Real About Distributed System Reliability

BOREDANDROID · 01 MAR 2026 · [SOURCE](#)

There is a lot of hype around distributed data systems, some of it justified. It's true that the internet has centralized a lot of computation onto services like Google, Facebook, Twitter, LinkedIn (my own employer), and other large web sites. It's true that this centralization puts a lot of pressure on system scalability for these companies. It's true that incremental and horizontal scalability is a *deep* feature that requires redesign from the ground up and can't be added incrementally to existing products. It's true that, if properly designed, these systems can be run with no *planned* downtime or maintenance intervals in a way that traditional storage systems make harder. It's also true that software that is explicitly designed to deal with machine failures is a very different thing from traditional infrastructure. All of these properties are critical to large web companies, and are what drove the adoption of horizontally scalable systems like [Hadoop](#), [Cassandra](#), [Voldemort](#), etc. I was the original author of Voldemort and have worked on distributed infrastructure for the last four years or so. So in-so-far as there is a "big data" debate, I am firmly in the "pro-" camp. But one thing you often hear is that this kind of software is more reliable than the traditional alternatives it replaces, and this just isn't true. It is time people talked honestly about this.

You hear this assumption of reliability everywhere. Now that scalable data infrastructure has a marketing presence, it has really gotten bad. Hadoop or Cassandra or what-have-you can tolerate machine failures then they must be unbreakable right? Wrong.

Where does this come from? Distributed systems need to partition data or state up over lots of machines to scale. Adding machines increases the probability that some machine will fail, and to address this these systems typically have some kind of replicas or other redundancy to tolerate failures. The core argument that gets used for these systems is that if a single machine has probability P of failure, and if the software can replicate data N times to survive $N-1$ failures, and if the machines fail *independently*, then the probability of losing a particular piece of data must be P^N . So for any desired reliability R and any single-node failure probability P you can pick some replication N so that $P^N < R$. This argument is the core motivation behind most variations on replication and fault tolerance in distributed systems. It is true that without this property the system would be hopelessly unreliable as it grew. But this leads people to believe that distributed software is somehow innately reliable, which unfortunately is utter hogwash.

Where is the flaw in the reasoning? Is it the dreaded Hadoop single points of failure? No, it is far more fundamental than that: the problem is the assumption that failures are independent. Surely no belief could possibly be more counter to our own experience or just common sense than believing that there is no correlation between failures of machines in a cluster. You take a bunch of pieces of identical hardware, run them on the same network gear and power systems, have the same people run and manage and configure them, and run the same (buggy) software on all of them. It would be incredibly unlikely that the failures on these machines would be independent of one another in the probabilistic sense that motivates a lot of distributed infrastructure. If you see a bug on one machine, the same bug is on all the machines. When you push bad config, it is usually game over no matter how many machines you push it to.

P^N is an upper bound on reliability but one that you could never, never approach in practice. For example Google has a fantastic paper that gives empirical numbers on system failures in Bigtable and GFS and reports empirical data on groups of failures that show rates several orders of magnitude higher than the independence assumption would predict. This is what one of the best system and operations teams in the world can get: your numbers may be far worse.

The actual reliability of your system depends largely on how bug free it is, how good you are at monitoring it, and how well you have protected against the myriad issues and problems it has. This isn't any different from traditional systems, except that the new software is far less mature. I don't mean this disparagingly, I work in this area, it is just a fact. Maturity comes with time and usage and effort. This software hasn't been around for as long as MySQL or Oracle, and worse, the expertise to run it reliably is much less common. MySQL and Oracle administrators are plentiful, but folks experience with, say, serious production Zookeeper operations knowledge are much more rare.

Kernel filesystem engineers say it takes about a decade for a new filesystem to go from concept to maturity. I am not sure these systems will be mature much faster—they are not easier systems to build and the fundamental design space is much less well explored. This doesn't mean they won't be *useful* sooner, especially in domains where they solve a pressing need and are approached with an appropriate amount of caution, but they are not yet by any means a mature technology.

Part of the difficulty is that distributed system software is actually quite complicated in comparison to single-server code. Code that deals with failure cases and is “cluster aware” is extremely tricky to get right. The root of the problem is that dealing with failures effectively explodes the possible state space that needs testing and validation. For example it doesn't even make sense to expect a single-node database to be fast if its disk system suddenly gets really slow (how could it), but a distributed system does need to carry on in the presence of single degraded machine because it has some many machines, one is sure to be degraded. These kind

of “semi-failures” are common and very hard to deal with. Correctly testing these kinds of issues in a realistic setting is brutally hard and the newer generation of software doesn’t have anything like the QA processes its more mature predecessors had. (If you get a chance get someone who has worked at Oracle to describe to you what kind of testing they do to a line of code that goes into their database before it gets shipped to customers). As a result there are a lot of bugs. And of course these bugs are on all the machines, so they absolutely happen together.

Likewise distributed systems typically require more configuration and more complex configuration because they need to be cluster aware, deal with timeouts, etc. This configuration is, of course, shared; and this creates yet another opportunity to bring everything to its knees.

And finally these systems usually want lots of machines. And no matter how good you are, some part of operational difficulty always scales with the number of machines.

Let’s discuss some real issues. We had a bug in [Kafka](#) recently that lead to the server incorrectly interpreting a corrupt request as a corrupt log, and shutting itself down to avoid appending to a corrupt log. Single machine log corruption is the kind of thing that should happen due to a disk error, and bringing down the corrupt node is the right behavior—it shouldn’t happen on all the machines at the same time unless all the disks fail at once. But since this was due to corrupt *requests*, and since we had one client that sent corrupt requests, it was able to sequentially bring down all the servers. Oops. Another example is [this Linux bug](#) which causes the system to crash after ~200 days of uptime. Since machines are commonly restarted sequentially this lead to a situation where a large percentage of machines went hard down one after another. Likewise any memory management problems—either leaks or GC problems—tend to happen everywhere at once or not at all. Some companies do public post-mortums for major failures and these are a real wealth of failures in systems that aren’t supposed to fail. [This paper](#) has an excellent summary of HDFS availability at Yahoo—they note how few of the problems are of the kind that high availability for the namenode would solve. This list could go on, but you get the idea.

I have come around to the view that the real core difficulty of these systems is operations, not architecture or design. Both are important but good operations can often work around the limitations of bad (or incomplete) software, but good software cannot run reliably with bad operations. This is quite different from the view of unbreakable, self-healing, self-operating systems that I see being pitched by the more enthusiastic NoSQL hypesters. Worse yet, you can’t easily buy good operations in the same way you can buy good software—you might be able to hire good people (if you can find them) but this is more than just people; it is practices, monitoring systems, configuration management, etc.

These difficulties are one of the core barriers to adoption for distributed data infrastructure. LinkedIn and other companies that have a deep interest in doing creative things with data have taken on the burden of building this kind of expertise in-house—we employ committers on Hadoop and other open source projects on both our engineering and operations team, and have done a lot of from-scratch development in this space where there was gaps. This makes it feasible to take full advantage of an admittedly valuable but immature set of technologies, and let's us build products we couldn't otherwise—but this kind of investment only makes sense at a certain size and scale. It may be too high a cost for small startups or companies outside the web space trying to bootstrap this kind of knowledge inside a more traditional IT organization.

This is why people should be excited about things like Amazon's DynamoDB. When DynamoDB was released, the company DataStax that supports and leads development on Cassandra released a feature comparison checklist. The checklist was unfair in many ways (as these kinds of vendor comparisons usually are), but the biggest thing missing in the comparison is that *you* don't run DynamoDB, Amazon does. That is a huge, huge difference. Amazon is good at this stuff, and has shown that they can (usually) support massively multi-tenant operations with reasonable SLAs, *in practice*.

I really think there is really only one thing to talk about with respect to reliability: continuous hours of successful production operations. That's it. In some ways the most obvious thing, but not typically what you hear when people talk about these systems. I will believe the system can tolerate (some) failures when I see it tolerate those failures; I believe it can run for a year without downtime when I see it run for a year without downtime. I call this empirical reliability (as opposed to theoretical reliability). And getting good empirical reliability is really, really hard. These systems end up being large hunks of monitoring, tests, and operational procedures with a teeny little distributed system strapped to the back.

You see this showing up in discussions of the CAP theorem all the time. The CAP theorem is a useful thing, but it applies more to system design than system implementations. A design is simple enough that you can maybe prove it provides consistency or tolerates partition failures under some assumptions. This is a useful lens to look at system designs. You can't hope to do this kind of proof with an actual system implementation, the thing you run. The difficulty of building these things means it is really unthinkable that these systems are, in actual reality, either consistent, available, *or* partition tolerant—they certainly all have numerous bugs that will break each of these guarantees. I really like this paper that takes the approach of actually trying to calculate the observed consistency of eventually consistent systems—they seem to do it via simulation rather than measurement, which is unfortunate, but the idea is great.

It isn't that system design is meaningless, it is worth discussing the system design as it does act as a kind of limiting factor on certain aspects of reliability and performance as the implementation matures and improves, but don't take it too seriously as guaranteeing *anything*.

So why isn't this kind of empirical measurement more talked about? I don't know. My pet theory is it has to do with the somewhat rigid and deductive mindset of classical computer science. This is inherited from pure math, and conflicts with the attitude in scientific disciplines. It leads to a preference for starting with axioms, and then proving various properties that follow from these axioms. This world view doesn't embrace the kind of empirical measurements you would expect to justify claims about reality (for another example see [this great blog post](#) on programming language productivity claims in programming language research). But that is getting off topic. Suffice it to say, when making predictions about how a system will work in the real world I believe in measurements of reality a lot more than arguments from first-principles.

I think we should insist on a little more rigor and empiricism in this area.

I would love to see claims in academic publication around practicality or reliability justified in the same way we justify performance claims—by doing it. I would be a lot more likely to believe an academic distributed system was practically feasible if it was run continuously under load for a year successfully and if information was reported on failures and outages. Maybe that isn't feasible for an academic project, but few other allegedly scientific academic disciplines can get away with making claims about reality without evidence.

More broadly I would like to see more systems whose *operation* is verifiable. Many systems have the ability to log out information about their state in a way that makes programmatic checking of various invariants and guarantees possible (such as consistency or availability). An example of such an invariant for a messaging system is that all the messages sent to it are received by all the subscribers. We actually measure the truth of this statement in real-time in production for our Kafka data pipeline for all 450 topics and all their subscribers. The number of corner-cases one uncovers with this kind of check, run through a few hundred use cases and a few billion messages per day is truly astounding. I think this is a special case of a broad class of verification that can be done on the running system that goes far far deeper than what is traditionally considered either monitoring or testing. Call it unit testing in production, or self-proving systems, or next generation monitoring, or whatever, but I think this kind of deep verification is something that makes turning the theoretical claims a design makes into measured properties of the real running system.

Likewise if you have a “NoSQL vendor” I think it is reasonable to ask them to provide hard information on customer outages. They don't need to tell you who the customer is, but they should let you know the observed real-life distribution of MTTF and MTTR they are able to

achieve, not just highlight one or two happy cases. Make sure you understand how they measure this, do they have automated test load that runs or just wait for people to complain? This is a reasonable thing for people paying for a service to ask for. To a certain extent if they provide this kind of empirical data it isn't clear why you should even *care* what their architecture is beyond intellectual curiosity.

Distributed systems solve a number of key problems at the heart of scaling large websites. I absolutely think this is going to become the default approach to handling state in internet application development. But no one benefits from the kind of irrational exuberance that currently surrounds the “big data” or nosql systems communities. This area is experiencing a boom—but nothing takes us from boom to bust like unrealistic hype and broken promises. We should be a little more honest about where these systems already shine and where they are still maturing.

Postscript: This blog seems to have resurfaced 7 years later. I actually still agree with much of it, though the dilemma is much less important. Distributed systems operations is hard, but you can now avoid it by just getting your distributed systems as a service. For example, Confluent offers Kafka as a service with a contractual uptime SLA. Ops remains hard, but for distributed data systems you can make the people who wrote the software deal with the problem, and focus your time on your special sauce.

Test Coverage Won't Save You

JENN COOPER · 09 JUN 2026 · [SOURCE](#)

A year ago, if a codebase accumulated three different ways to do the same thing, somebody would usually notice.

A reviewer might leave a comment. A senior engineer would mutter something about "yet another helper," and eventually the team would clean it up. Maybe they'd write a short doc. People would learn, collectively, that this was not the way.

This was not a perfect system. Human review can be slow. People miss things. Pattern docs get stale. Engineers develop opinions that are 70% wisdom and 30% scar tissue.

Still, the friction did something useful. It kept codebases from drifting too quickly.

Coding agents change that.



"Now, this is a room with electricity. But it has too much electricity."

Once agents are writing a meaningful share of code, the question is no longer just "can we get working code faster?" Obviously, yes. We can. Sometimes hilariously so.

The more interesting question is: what does the codebase teach the next agent?

Because it does teach.

Every merged PR becomes a precedent. If the repository contains one clear pattern, the next agent has a decent shot at following it. If it contains three slightly different patterns, the next agent may extend one, combine two, or invent a fourth for reasons that are, in technical terms, vibes.

The funhouse builds itself

Coding agents, like some of us, have a chaotic inner goth teenager.

Their non-deterministic nature means they can be inconsistent, often in surprising ways. They can generate the weird little leap that solves the problem, or they can generate novelty where nobody asked for it.

The dangerous thing is that most of the code is fine, if you look at it in isolation.

A PR passes tests. The implementation makes sense. The file is clean enough. You look at the diff and think, yeah, sure, that seems okay.

But zoom out, and the codebase as a whole has started to get weird.

- three near-duplicate utilities

- multiple data-fetching patterns
- parallel abstractions solving almost the same problem

The codebase remains locally **cohesive**, while slowly losing global **coherence**.

That matters more than it used to, because even more than humans, agents are responsive to the signals around them. A clear codebase gives them clear precedent. A muddled codebase gives them confusion.

You can put principles in `CLAUDE.md` or docs, but they're... advisory. They compete with task instructions and inline comments for attention, get summarized when context windows compact, and depend on the agent attending to the right instruction at the right moment.

If your codebase contains three slightly different ways to solve a problem, the next agent has to infer which one is canonical.

From the agent's point of view, this is not irrational. The repository gave it mixed evidence.

The scary outcome is not code that obviously fails. It is code that superficially keeps working, while the shape of the system increasingly gets worse.

Tests are nice

Most discussions about AI-native development jump from this problem – agents' tendency to accumulate tech debt – directly to tests.

And yes, across the industry, teams are writing dramatically more tests than they ever have. Agents have made high test coverage affordable in a way it never used to be.

Recently, Garry Tan argued that the primary way to keep AI agents on track is 90% test coverage:

| *Tests are the ratchet. 90% coverage, every PR, no exceptions.*

And indeed, test coverage is the simplest kind of ratchet: a mechanism that allows motion in one direction only, like a socket wrench that turns the bolt forward but never lets it spin back. Once a test locks in a behaviour, it becomes difficult to accidentally regress.

But it's worth being specific about what tests actually do.

Unit tests check that a function still returns what it returned before. Integration tests check that pieces still wire together the same way. E2E tests reach further: they check whether the product still does what it used to, and the assertions you prioritize there should be an important act of human judgment.

But they all share the same basic shape: tests verify that code does what it did before.

Whether what it did was even the *right way to do it* is a separate question.

Tests ratchet behavioural sameness.

Meanwhile docs and evals can pin down reasoning and behaviour bars, which is sometimes more important. But none of these is directly looking at the *shape* of the system.

90% coverage of good patterns

A coverage ratchet only improves the codebase if the patterns it's locking in are already good ones.

If a questionable pattern already exists in the code, the next agent is more likely to extend it. The tests generated around that implementation only reinforce it further.

Over time, high coverage can unintentionally fossilize architectural drift, rather than prevent it in the first place.

At first, especially during prototyping, this can be easy to miss, because nothing looks obviously broken. Coverage stays high. The system works. But over time, certain parts of the codebase become strangely resistant to cleanup.

Tests that protect the wrong behaviour create drag against coherence. The harder you push for coverage, the more deeply you can lock in the shape you happened to start with.

This isn't a criticism of tests. Tests are necessary. But tests primarily validate behaviour, while many of the emerging failure modes in agent-generated systems, especially as they grow, are failures of *shape*.



"Uh, I wouldn't take it down if I were you. It's a load-bearing poster."

Fitness functions protect the shape of the system

Unit tests ask whether the code still does what it did before. Fitness functions ask a different question: is the system still shaped the way we want?

Most codebases already have a few simple versions of this. Linters, type checks, and import-boundary rules all encode some idea of acceptable shape, at the micro level.

But in agentially-maintained codebases, we need an additional kind of macro fitness function. It has a couple of important jobs:

- catch bad agent choices at the moment of work
- redirect agents with the right amount of friction for the violation

At Forestwalk, for example, we have a fitness check that prevents tool handlers from calling back into our AI services. Tool handlers run inside the agent's own loop, so any handler that kicks off more inference can stack model calls inside model calls and freeze a live meeting. Agents sometimes accidentally introduce this pattern, so our fitness check flags it and ensures it's not committed.

Other fitness tests are ratchets that apply pressure to measurable properties – heuristics for shape like file size, bundle growth, prompt size, import fanout, and dependency count.

For these, the ceiling starts at the current value (e.g. 837kb) and flags to agents and humans if a PR would unduly increase it. For instance, when an agent adds an unnecessary new dependency that blows up our frontend bundle, the check fails.

The important part isn't just the limit itself. It's the “just in time” agent coaching.

At first, we found agents would sometimes over-optimize against whatever metric the checks exposed, with great resourcefulness and in ways we did not want. But refining the flags to give common sense guidance helped steer the agents into considering tradeoffs correctly.

For example, an agent might see this just in time:

```
🔴🔴🔴🔴🔴🔴 Failed Checks: 1 🔴🔴🔴🔴🔴
```

```
FAIL src/frontendBundleBudget.test.ts > Frontend bundle size budget
AssertionError: Frontend bundle grew +29 KiB (700 -> 729 KiB; fails a
```

This ratchet is review pressure, not a mandate to shrink the number a

Diagnose first:

- Is the growth accidental (dependency, duplicate/dead code, broad im
- Does the added JS unlock user-visible value?
- Can cleanup reduce it without hurting UX, accessibility, tests, or

Quality-preserving fixes:

- remove duplicated rendering paths or dead production code
 - move repeated styling or static configuration out of lazy-loaded Ja
 - lazy-load rarely used flows instead of loading them on the main pat
- (more examples here)

Do not fix this by:

- omitting requested scope or deferring needed UI work
 - removing aria-label/title text or visible affordances
 - deleting helpful empty/error-state copy
- (more examples here)

For intentional growth, (procedure here) and explain why in the PR.

If you simply tell agents to "reduce bundle size," they will sometimes do naive things like:

- delete empty-state copy
- replace styled controls with browser-native ones
- inline dependency code to avoid import costs
- remove accessibility affordances

Instead, the idea is to not just explain what a given fitness test is protecting, but also explain what is *not allowed* to be sacrificed in the process, and do so at the moment the agent is working.

In contrast, the same guidance in `CLAUDE.md` is often skimmed past or applied inconsistently by agents. A just-in-time check, on the other hand, creates desired friction between the agent and a finished task. Where humans get tired and occasionally creative about routing around friction like this, agents are quite willing to schlep through feedback and loop until the check is passing.



"Come on in, it's your master bedroom!"

What the checks can't see

Fitness functions are good at catching things precise enough to encode.

That is useful, but coherence is not always that crisp. Some drift is obvious only in relation to the rest of the codebase: this new thing is too close to an existing thing, this test is protecting a pattern we no longer want, this change is making the next change harder.

So in addition, we use a few non-deterministic checks on top of the deterministic ones.

One runs before a PR is opened. It scans the proposed change against the existing codebase, and asks whether the agent has introduced or expanded a competing pattern. When it finds one, it points to the existing pattern, explains why the new one looks redundant, and asks the agent to either extend the original or justify why this case is genuinely different.

Most of the time, the agent course-corrects quickly.

We run a similar check again at the PR review level, where it has more context and can catch subtler pattern drift.

Another check runs nightly and looks for tests that have fossilized patterns we have since decided against. Its mission is to find tests that protect old implementation choices that, if left alone, would teach future agents to preserve patterns we no longer want.

These checks are not magic. They miss things. They can be noisy. They still need human judgment around them.

But the goal is not to eliminate judgment. The goal is to move more human judgment into places where it can be highest leverage.

Making coherence the easy path

Fitness ratchets are not the whole answer to maintaining coherence in agent-generated codebases.

Fitness functions are one layer. Pattern drift checks are another. Product verification, code review, evals, documentation, repository structure, and the actual taste of the humans steering the system all matter too.

However, it's worth investing regularly in the early layers: the ones closest to the moment an agent is making an implementation choice.

That is where we have found these types of fitness functions useful. They do not solve coherence by themselves, but they add friction in the right places. Bad patterns get interrupted earlier. Good patterns have a better chance of becoming precedent.

The same feedback loop that spreads bad patterns can be used to spread good ones.

So there's hope. When you see this pattern work, it soothes some of the worry we all have about agents causing chaos. They're a lot more useful when the codebase gives them an obvious next move. The right pattern is easier to find. The weird fourth version of the thing has a harder time looking reasonable.

It's not perfect. Keeping a growing codebase coherent still requires human judgment, architectural intuition, and social coordination. The system still needs to be built.

But that system, itself, increasingly feels like the work.

If the codebase trains the next agent, then the job is not just to write better code.

It is to build the environment where better code is the obvious thing to write.

A non-anthropomorphized view of LLMs

HALVAR.FLAKE · 14 SEP 2025 · [SOURCE](#)

In many discussions where questions of "alignment" or "AI safety" crop up, I am baffled by seriously intelligent people imbuing almost magical human-like powers to something that - in my mind - is just MatMul with interspersed nonlinearities.

In one of these discussions, somebody correctly called me out on the simplistic nature of this argument - "a brain is just some proteins and currents". I felt like I should explain my argument a bit more, because it feels less simplistic to me:

The space of words

The tokenization and embedding step maps individual words (or tokens) to some vectors. So let us imagine for a second that we have in front of us. A piece of text is then a path through this space - going from word to word to word, tracing a (possibly convoluted) line.

Imagine now that you label each of the "words" that form the path with a number: The last word with 1, counting forward until you hit the first word or the maximum context length . If you've ever played the game "Snake", picture something similar, but played in very high-dimensional space - you're moving forward through space with the tail getting truncated off.

The LLM takes your previous path into account, calculates probabilities for the next point to go to, and then makes a random pick into the next point according to these probabilities. An LLM instantiated with a fixed random seed is a mapping of the form .

In my mind, the paths generated by these mappings look a lot like [strange attractors](#) in dynamical systems - complicated, convoluted paths that are structured-ish.

Learning the mapping

We obtain this mapping by training it to mimic human text. For this, we use approximately all human writing we can obtain, plus corpora written by human experts on a particular topic, plus some automatically generated pieces of text in domains where we can automatically generate and validate them.

Paths to avoid

There are certain language sequences we wish to avoid - because the sequences these models generate try to mimic human speech in all it's empirical structure, but we feel that some of the things that humans have empirically written are very undesirable to be generated. We also feel that a variety of other paths should ideally not be generated, if - when interpreted by either humans or other computer systems - undesirable results arise.

We can't specify strictly in a mathematical sense which paths we would prefer not to generate, but we can provide examples and counterexamples, and we try to hence nudge the complicated learnt distribution away from them.

"Alignment" for LLMs

Alignment and safety for LLMs mean that **we should be able to quantify and bound the probability with which certain undesirable sequences are generated**. The trouble is that we largely fail at describing "undesirable" except by example, which makes calculating bounds difficult.

For a given LLM (without random seed) and sequence, it is trivial to calculate the probability of the sequence to be generated. So if we had a way of somehow summing or integrating over these probabilities, we could say with certainty "this model will generate an undesirable sequence once every N model evaluations". We can't, currently, and that sucks, but at the heart, this is the mathematical and computational problem we'd need to solve.

The surprising utility of LLMs

LLMs solve a large number of problems that could previously not be solved algorithmically. NLP (as the field was a few years ago) has largely been solved.

I can write a request in plain English to summarize a document for me and put some key datapoints from the document in a structured JSON format, and modern models will just do that. I can ask a model to generate a children's book story involving raceboats and generate illustrations, and the model will generate something that is passable. And much more, all of which would have seemed like absolute science fiction 5-6 years ago.

We're on a pretty steep improvement curve, so I expect the number of currently-intractable problems that these models can solve to keep increasing for a while.

Where anthropomorphization loses me

The moment that people ascribe properties such as "consciousness" or "ethics" or "values" or "morals" to these learnt mappings is where I tend to get lost. We are speaking about a big recurrence equation that produces a new word, and that stops producing words if we don't crank the shaft.

To me, wondering if this contraption will "wake up" is similarly bewildering as if I was to ask a computational meteorologist if he isn't afraid of his meteorological numerical calculation will "wake up".

I am baffled that the AI discussions seem to never move away from treating a function to generate sequences of words as something that resembles a human. Statements such as "an AI agent could become an insider threat so it needs monitoring" are simultaneously unsurprising (you have a randomized sequence generator fed into your shell, literally **anything** can happen!) and baffling (you talk as if you believe the dice you play with had a mind of their own and could decide to conspire against you).

Instead of saying "we cannot ensure that no harmful sequences will be generated by our function, partially because we don't know how to specify and enumerate harmful sequences", we talk about "behaviors", "ethical constraints", and "harmful actions in pursuit of their goals". All of these are anthropocentric concepts that - in my mind - do not apply to functions or other mathematical objects. And using them muddles the discussion, and our thinking about what we're doing when we create, analyze, deploy and monitor LLMs.

This muddles the public discussion. We have many historical examples of humanity ascribing bad random events to "the wrath of god(s)" (earthquakes, famines, etc.), "evil spirits" and so forth. The fact that intelligent highly educated researchers talk about these mathematical objects in anthropomorphic terms makes the technology seem mysterious, scary, and magical.

We should think in terms of "this is a function to generate sequences" and "by providing prefixes we can steer the sequence generation around in the space of words and change the probabilities for output sequences". And for every possible undesirable output sequence of a length smaller than , we can pick a context that maximizes the probability of this undesirable output sequence.

A much clearer formulation, which helps more clearly articulate the problems to solve.

Why many AI luminaries tend to anthropomorphize

Perhaps I am fighting windmills, or rather a self-selection bias: A fair number of current AI luminaries have self-selected by their belief that they might be the ones getting to AGI - "creating a god" so to speak, the creation of something like life, as good as or better than humans. You are more likely to choose this career path if you believe that it is feasible, and that current approaches might get you there. Possibly I am asking people to "please let go of the belief that you based your life around" when I am asking for an end to anthropomorphization of LLMs, which won't fly.

Why I think human consciousness isn't comparable to an LLM

The following is uncomfortably philosophical, but: In my worldview, humans are dramatically different things than a function. For hundreds of millions of years, nature generated new versions, and only a small number of these versions survived. Human thought is a poorly-understood process, involving enormously many neurons, extremely high-bandwidth input, an extremely complicated cocktail of hormones, constant monitoring of energy levels, and millions of years of harsh selection pressure.

We understand essentially nothing about it. In contrast to an LLM, given a human and a sequence of words, I cannot begin putting a probability on "will this human generate this sequence".

To repeat myself: To me, considering that any human concept such as ethics, will to survive, or fear, apply to an LLM appears similarly strange as if we were discussing the feelings of a numerical meteorology simulation.

The real issues

The function class represented by modern LLMs are very useful. Even if we never get anywhere close to AGI and just deploy the current state of technology everywhere where it might be useful, we will get a dramatically different world. LLMs might end up being similarly impactful as electrification.

My grandfather lived from 1904 to 1981, a period which encompassed moving from gas lamps to electric, the replacement of horse carriages by cars, nuclear power, transistors, all the way to computers. It also spanned two world wars, the rise of Communism and Stalinism, almost the entire lifetime of the USSR and GDR etc. The world on his birth looked nothing like the world when he died.

Navigating the dramatic changes of the next few decades while trying to avoid world wars and murderous ideologies is difficult enough without muddying our thinking.